

Ingeniería del Software 3

Clase 5 · Testing en el pipeline: unit tests, coverage y el umbral que frena un merge

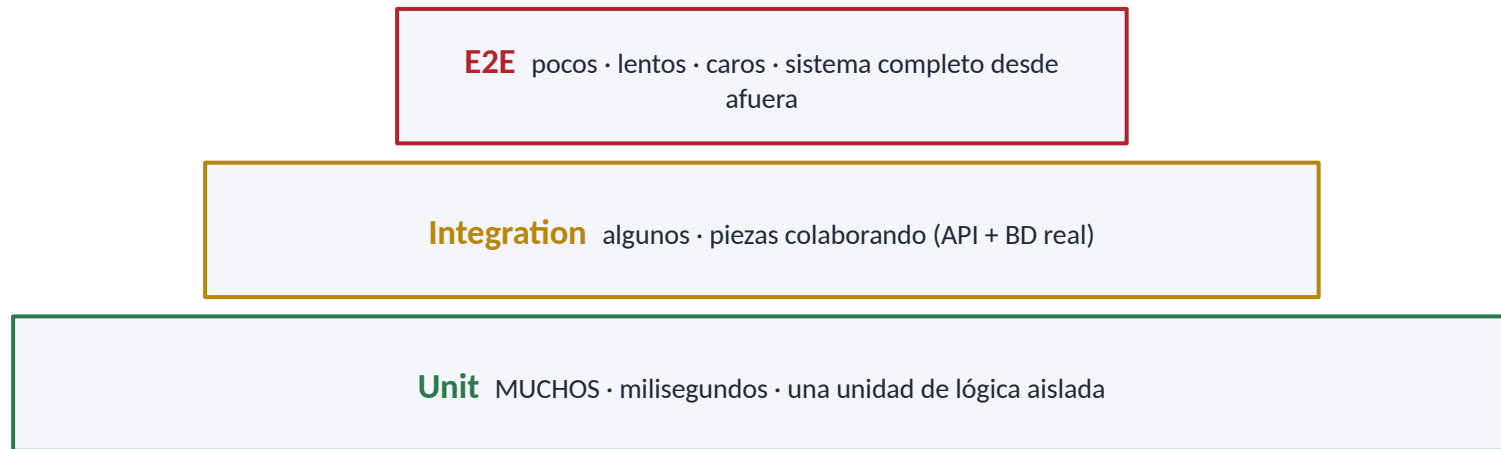
UCC · Ingeniería en Sistemas · 2026 · Ing. Ariel Schwindt

Lo que vas a ver en este video

- Se trabaja sobre TU app del semestre, en el mismo repo — y sobre el pipeline del TP4, que tiene que estar verde
- Si el TP4 no está terminado, ésa es la prioridad: el TP5 se apila encima, no lo reemplaza

Bloque	De qué se trata
La teoría	La pirámide · cómo se escribe un test que se entiende · los mocks · cobertura y sus límites · cómo comprobar que tus tests verifican (mutantes) · análisis estático · la calidad como requisito para mergear
El cierre	Lo que queda cuando el pipeline termina — el puente al TP6, donde pasa a ENTREGAR
El práctico	Qué tenés que entregar y con qué se evalúa
Y después	El paso a paso completo, en el video del TP5

La pirámide de testing: qué testear y cuánto



- **La forma importa: invertirla** —el «cono de helado»— **da una suite lenta, frágil, y que cuando falla no te dice DÓNDE**
- Los tres, en tu app: una función que calcula un precio → UNITARIO · tu API consultando la base de verdad → INTEGRACIÓN · un navegador que carga la pantalla y hace clic → E2E
- **Hoy construimos la BASE: escribir esos tests y medir cuánto de tu código ejercitan** · las e2e llegan en el TP7, cuando haya un entorno desplegado donde correrlas

Anatomía de un unit test: AAA

```
[Fact] // así se marca un test en xUnit
public void UnDescuentoDel10SobreMilPesos_Da900()
{
    // Arrange: los datos de entrada
    var precio = 1000;
    var descuento = 10;

    // Act: la operación que se está probando — una sola
    var final = Precios.ConDescuento(precio, descuento);

    // Assert: el resultado que se espera
    Assert.Equal(900, final);
}
```

- El NOMBRE describe el comportamiento, no el método: cuando falle a las 3 AM, el nombre ES el diagnóstico
- 🏗️ ¿Nunca escribiste un test? El §3.0 de la guía arma la suite paso a paso
- Un comportamiento por test: si verifica cinco cosas y falla, no sabés cuál se rompió
- Mismo comportamiento con varios datos: se escribe UNA vez con su lista — [Theory]+[InlineData], o it.each en el front

¿Y si mi lógica depende de la base de datos?

✗ Acoplado: la dependencia se fabrica adentro

```
class Servicio {  
    var repo = new RepoSql();  
}
```

clavado



Postgres

✓ Inyección de dependencias: no la fabrica, la RECIBE — y por ahí entra el mock

```
class Servicio {  
    Servicio(IRepo repo) { ... }  
}
```

IRepo (interfaz)



RepoSql • producción

RepoEnMemoria • tests

Si algo es difícil de testear, sospechá del diseño antes que del test.

Y el caso más feliz: la LÓGICA PURA no necesita nada. Recibe valores, devuelve un valor, no toca base ni red ni pantalla:

```
ConDescuento(1000, 10) → 900 // cero mocks
```

Igual el TP te pide UN mock sí o sí

Así se escribe un test con mock

La regla: al registrar un usuario se manda UN mail de bienvenida. No queremos mandar mails de verdad en cada test.

```
[Fact]
public void AlRegistrar_SeMandaUnMailDeBienvenida()
{
    // Arrange: un mock del servicio de mails, en vez del real
    var mails = new Mock<IServicioMails>();
    var servicio = new RegistroService(mails.Object);    // ← entra por acá

    // Act
    servicio.Registrar("ana@ucc.edu.ar");

    // Assert: lo que se verifica es la INTERACCIÓN, no un valor devuelto
    mails.Verify(m => m.Envia("ana@ucc.edu.ar"), Times.Once);
}
```

- Fijate dónde entra: el servicio RECIBE su dependencia (la filmina anterior). Con un new adentro, este test no se puede escribir
- El Mock<> sale de una LIBRERÍA que se instala: en .NET es Moq (dotnet add package Moq); en vitest, vi.fn(). El §3.0 de la guía tiene el comando
- Tres nombres que vas a leer: stub (te da datos), mock (verifica que algo se llamó — éste), fake (una implementación simplificada, como un repositorio en memoria). En la práctica se les dice «mock» a los tres, y las librerías se llaman así

Code coverage: qué mide

Una función. Un test, con edad = 20. Dos formas de contar lo mismo:

① por LÍNEAS

```
string Clasificar(int edad)
{
    if (edad >= 18)      ✓
        return "mayor";  ✓
    else
        return "menor";  ✗
}
```

4 de 5 líneas → 80%

② por CAMINOS (ramas)

edad >= 18 → "mayor"

✓ lo probó tu test

edad < 18 → "menor"

✗ no pasó nadie

1 de 2 caminos → 50%

80% suena bien — y estás probando la MITAD de tu lógica. Por eso se mira el de caminos.

- Coverage = qué % de tu código SE EJECUTÓ mientras corrían los tests. Lo mide la herramienta de tu stack: en el ejemplo, coverlet (viene con xUnit) y vitest. Otro stack, otro nombre; el mismo par de números

Coverage $\neq$ calidad: la trampa de la métrica

```
[Fact]
public void CoberturaSinVerdad()
{
    Precios.ConDescuento(1000, 10);    // ¿coverage? 100%. ¿Asserts? CERO.
}
```

La cobertura mide **EJECUCIÓN**, no **VERIFICACIÓN**: ese test recorre el método entero y no comprueba nada.

El número que ves	Qué podés concluir
Cobertura BAJA	Señal confiable: hay código que ningún test ejercita — ahí puede esconderse cualquier cosa
Cobertura ALTA	No concluye nada por sí sola: pudo conseguirse con tests que ejecutan sin verificar

- **Ley de Goodhart:** cuando una medida se convierte en objetivo, deja de ser una buena medida. Un equipo con un bonus atado al 90% de cobertura llega al 90% — con tests que no comprueban nada
- Por eso el TP no te impone un número: te pide EL TUYO, con su justificación (alcance, exclusiones, porqué). Un 80% que podés defender vale más que un 95% que nadie eligió

Mutantes: ¿tus tests verifican, o sólo ejecutan?

La cobertura contesta qué se EJECUTA. Esto contesta la otra mitad: de eso que se ejecuta, ¿qué está realmente comprobado?
— un mutante es tu código con UN cambio hecho a propósito

tu código: el descuento, ahora con el tope

```
if (descuento > 50) rechazar();
```



el mutante: un solo carácter

```
if (descuento >= 50) rechazar();
```

*ahora un descuento
de exactamente 50
se rechaza mal*

1 • el que tenés hoy — es un CASO DE ERROR: espera un rechazo

```
// 60 se pasa del tope: tiene que rechazarlo  
Assert.Throws<ArgumentException>(  
    () => ConDescuento(1000, 60) );
```

2 • el test que FALTA — y que lo mata

```
// el borde exacto: 50 SÍ se acepta  
Assert.Equal( 500, ConDescuento(1000, 50) );
```

Pasa con las DOS versiones → SOBREVIVió

60 también es «mayor o igual a 50»: el test no distingue

Con tu código pasa ✓ • con el mutante FALLA ✗

MUTANTE MUERTO

el mutante rechaza el 50, y el test esperaba 500

El mutante no te dio un test más: te dijo CUÁL te faltaba — el del BORDE. La cobertura no podía: esa línea del if ya se ejecutaba.

- Mucho más difícil de inflar que la cobertura: aquel test que ejecutaba todo y no comprobaba nada llena la cobertura entera y no mata a estos mutantes
- Corre en tu máquina, no en el pipeline: cada mutante necesita su propia corrida de tests (sólo los que cubren esa línea, no la suite entera) — de minutos a horas, contra los segundos de la cobertura. El práctico NO te pide correr mutantes: esto es para que entiendas el límite del número

¿Dónde corren los tests? Una sola receta, no dos

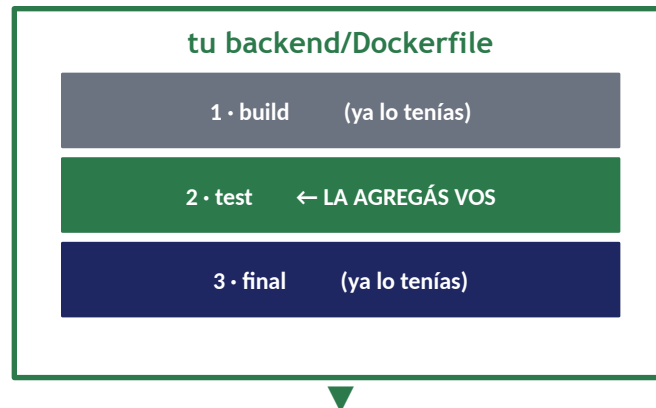
✗ DOS RECETAS



⚡ el día que una cambia, divergen

«el pipeline te dice que está todo bien,
y está mintiendo sin saberlo»

✓ UNA SOLA RECETA



verificás con la MISMA receta con la que se arma lo que desplegás

```
# backend/Dockerfile – NO es un archivo nuevo: es el del TP2, con una etapa más EN EL MEDIO
# (asume que tu etapa de compilar se llama build; si le pusiste otro nombre, usá ése)
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
...
# ¡ ESTAS DOS INSTRUCCIONES son lo que agregás vos
FROM build AS test
ENTRYPOINT ["dotnet", "test", "Backend.sln", "--logger", "trx;LogFileName=tests.trx", \
    "--results-directory", "/out"]
# ¡ ésta ya estaba y no se toca: es la que queda ÚLTIMA
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
```

¿Y cómo hace el pipeline para correr esos tests?

No hay un job nuevo ni un archivo nuevo: son cuatro pasos más, al final del job que ya tenías.

job build-backend — lo que ya tenías

- 1 • checkout: traer tu código
- 2 • construir la imagen (push: false)

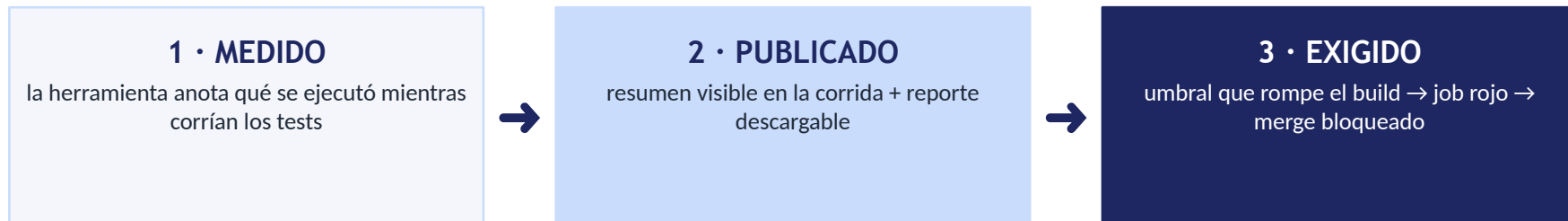
job build-backend — al terminar el §3.2

- 1 • checkout: traer tu código
- 2 • construir la imagen (push: false)
- 3 • construir la etapa test ← nuevo
- 4 • docker run: correr los tests ← nuevo
- 5 • armar el reporte y el resumen ← nuevo
- 6 • publicar el reporte ← nuevo

```
- name: Construir la etapa de tests
  uses: docker/build-push-action@v7      # la MISMA del TP4
  with:
    context: ./backend
    target: test      # frená en la etapa de tests, no sigas hasta final
    load: true        # dejala en el Docker de la máquina, para poder arrancarla
    tags: backend-test:ci
- name: Correr los tests
  run: |
    docker run --rm -v "$PWD/TestResults:/out" backend-test:ci \
      --collect:"XPlat Code Coverage"
```

Si un test falla, el docker run termina con error → el paso falla → el job se pone rojo. Nada nuevo que configurar: es la misma cadena del TP4.

La escalera: medido → publicado → exigido



La cadena que te importa, y que ya conocés: los tests salen con error → el paso falla → el job se pone ROJO → el merge queda bloqueado.

Lo MEDIDO y lo PUBLICADO se resuelven en la corrida (§3.2). Lo EXIGIDO son DOS números que elegís vos, y van en DOS lugares distintos — acá abajo los dos, porque el del backend es el que más se olvida:

El gate del TP4 ya sabía bloquear: el umbral sólo le agrega una condición más que vigilar.

FRONTEND — en vite.config.js (sólo se aplica si la corrida pide --coverage)

```
test: {
  coverage: {
    include: ['src/TU-CARPETA/**'], // si no matchea:
                                // 0 archivos y verde falso
    thresholds: { lines: 80, branches: 80 } // TU número
  }
}
```

BACKEND — en el ENTRYPOINT de tu etapa de tests (paquete coverlet.msbuild)

```
ENTRYPOINT ["dotnet", "test", "Backend.sln",
  "/p:CollectCoverage=true",
  "/p:Threshold=80",
  "/p:ThresholdType=line", // ● CADA UNO con su coma:
  ...]                    // pegados, dotnet los IGNORA
```

Análisis estático: leer el código sin ejecutarlo

Capa	Herramienta	Qué atrapa
Compilador + linters	Roslyn analyzers (los del compilador de C#) · ESLint	Tipos, sintaxis, errores comunes, estilo
Seguridad (SAST)	CodeQL — gratis en repo público	Vulnerabilidades: inyección SQL, XSS, secretos, path traversal — cada una con su categoría OWASP
Mantenibilidad	SonarQube Cloud	Bugs probables · duplicación · complejidad — deuda técnica MEDIDA, con estimación de esfuerzo

- Tests = análisis DINÁMICO (ejecutan) · esto = ESTÁTICO (lee el fuente) — atrapan clases de problemas DISTINTAS
- Los tests solo verifican lo que a alguien se le ocurrió testear; el análisis estático revisa TODO el código, siempre
- ¿Por qué en el pipeline? La regla del semestre: lo que no corre automático en cada cambio, se degrada
-  ESTO NO SE PIDE EN EL TP5. La capa de SEGURIDAD se practica en el TP9 (su Tarea 3): CodeQL en cada PR + 2 hallazgos analizados con su categoría OWASP. La de MANTENIBILIDAD queda como concepto — hoy la ves para saber que existe y qué pregunta contesta

Quality gate: cuando la calidad, y no la compilación, frena un merge

*Quality gate = una condición medible que hay que cumplir para avanzar.
La tuya esta semana: la cobertura, arriba del número que declaraste vos.*

	Qué lo pone en ROJO	Qué NO ve
Gate del TP4	El código no compila · la imagen no se arma	Si el código anda, y si está probado
Gate del TP5	La cobertura quedó abajo de TU número — compilando bien y con todos los tests en verde	Si el requisito se entendió bien

- Un job en rojo, POR SÍ SOLO, no impide nada: hay que declarar el check como required (Settings → Branches, el mismo lugar del TP1 y el TP4). Y no hay ningún check nuevo — son los DOS del TP4, que ahora se pueden poner rojos por otra razón
- Que el número sea TUYO no lo hace más blando, lo hace más exigible: un umbral impuesto desde afuera se apaga el día que molesta. La regla que hace adoptable a cualquier gate es la misma — «lo que tocás, lo dejás limpio»

El gate ampliado: los tres guardianes de main

TP1

La revisión (humana, con IA al lado)

El único que puede ver que el REQUISITO se entendió mal — porque ese dato no está en el código. Trabajando solo no te pide aprobaciones: el que mira sos vos

TP4

La imagen se construye

Compila y la imagen se arma — nada más, y no es poco

TP5

Umbral de coverage

Tu código entra EJERCITADO por tests — el umbral que elegiste vos, frenando el merge
— y el cuarto, el que todavía NO tenés —

TP9

Análisis estático

El cuarto, que todavía no tenés: lee tu código sin ejecutarlo y busca lo que a nadie se le ocurrió testear

El de esta semana NO agrega ningún check nuevo: son los DOS del TP4 (Settings → Branches, el mismo lugar de siempre), que desde hoy también se ponen rojos por cobertura. Lo que cambió no es la lista de guardianes: es de qué se dan cuenta

Qué QUEDA cuando el pipeline termina

Un pipeline que no produce artefacto no es integración continua: es compilación.

La condición no está en ningún «if»: está en el
ORDEN de los pasos

1 • construir

2 • correr los tests

3 • cobertura

4 • publicar la imagen 

X si algo falla acá, el job
muere y lo de abajo
NUNCA llega a correr

Por qué el registry es confiable

1 • Nada entra a main sin los guardianes automáticos en verde

2 • Sólo lo que entra a main se publica

⇒ Todo lo que el PIPELINE publica pasó la verificación completa

Esto lo construís en el TP6, donde el artefacto es lo que se PROMUEVE de un entorno a otro. Acá sólo te queda la idea: el pipeline verifica para poder entregar.

Esta semana NO se te pide publicar nada: el TP5 cierra en el quality gate. Esta filmina es el puente — la semana que viene el pipeline pasa de verificar a entregar.

TP5 — Calidad automatizada: qué tenés que hacer

El escenario: se coló un bug por código que nadie testeaba, y el pipeline estaba en verde. Esta semana la calidad pasa a ser medida y exigida.

1

Suite de unit tests significativa

≥8 en el back y ≥4 en el front — y en LOS DOS: un parametrizado, un caso de error y UN MOCK

2

Coverage medido y publicado

Back y front en el pipeline, resumen visible + reporte descargable, y TU umbral justificado rompiendo el build

3

El umbral bloqueando un merge

Los checks del TP4 como required + la secuencia documentada: PR que COMPILA y con los tests en verde, frenado por cobertura → los tests que faltaban → verde → merge



Se corrige el LOGRO, no la herramienta: los ejemplos son .NET + vitest porque un ejemplo tiene que estar escrito en algo. La tabla «Tu stack, de un vistazo» (§3 de la guía) traduce las siete cosas que se piden a Java, Python y JS.



¿Tu app es un CRUD pelado? Entonces no tenés qué testear: agregale reglas de negocio propias y contalo en decisiones.md. Era el criterio del TP2 al elegir la app.

TP5 — Entregables

Al formulario va UNA sola dirección: la de tu repo. Todo lo demás vive en decisiones.md — y cada decisión va CON el enlace que la prueba al lado.

Dónde	Qué tiene que estar ahí
El repo (al formulario)	Tu suite de tests · el workflow extendido · los Pull Requests, con el que quedó frenado por el gate
decisiones.md abajo: la sección del TP5	Qué lógica elegiste testear y por qué ÉSA · tu umbral: número, alcance y exclusiones · por qué coverage alto NO garantiza calidad, con TU ejemplo · tu Pull Request bloqueado: qué check se puso en rojo y qué escribiste para arreglarlo · el ejercicio de la rama sin cubrir · uso de IA

- ¿URL de qué? De la CORRIDA o del PULL REQUEST concretos, no del repo: `.../actions/runs/<id>` · `.../pull/<n>`. Son cuatro — tu cobertura en un run · el run ROJO por umbral con el número en el log · el PR frenado · y ese mismo PR, que muestra la secuencia hasta el merge
- En la defensa (P2) mostrás: TU test señalando AAA, TU umbral con su porqué, TU PR bloqueado por el gate · Pesos de siempre: 25% técnico · 25% decisiones · 50% defensa

Cierre — la calidad ya no es opinión

1

TP5 esta semana: la suite, la cobertura y el umbral que frena

La secuencia rojo-POR-COBERTURA→los tests que faltaban→verde→merge es tu evidencia central

2

Escribí tu umbral HOY en decisiones.md

Número + sobre qué métrica + alcance + exclusiones + porqué — hay que poder defenderlo

3

Próxima clase: CD — environments, aprobaciones y deployment

Tu pipeline deja de solo verificar: empieza a ENTREGAR y a DESPLEGAR